

Document #: P1901R0

Date: 2019-10-02

Reply-to: Daryl Haresign <cpp@daryl.haresign.com>

Audience: Library Evolution Working Group

Enabling the Use of `weak_ptr` as Keys in Unordered Associative Containers

Contents

- [Abstract](#)
- [Motivation](#)
- [Proposal](#)
- [Alternatives Considered](#)
- [Wording](#)
- [References](#)
- [Acknowledgements](#)

Abstract

This paper proposes the addition of ownership based hashing and equality, to enable the use of `weak_ptr` as keys in unordered associative containers.

Motivation

I recently found myself wanting to store a collection of `weak_ptr` objects. I neither had a separate key nor any requirements on the ordering of the objects. It seemed that using an `unordered_set` would be a good fit. It quickly became apparent, however, that it's simply not possible; there is no way to generate a stable hash or compare `weak_ptr` objects without support from the Standard Library.

Also relevant is the discussion in LWG Issue 1406, which labelled this as not a defect, and recommended a paper be brought forward to propose the addition.

Proposal

`owner_hash` and `owner_equal`

The obvious, and in my opinion best, option is to follow in the footsteps of `owner_less`, which can be used to allow `weak_ptr` keys in ordered associative containers.

N2637 was the paper through which `owner_less` was introduced into the standard.

The following methods and class templates would be added:

- `size_t shared_ptr::owner_hash() const noexcept;`
- `bool shared_ptr::owner_equal(...) const noexcept;`

- `size_t weak_ptr::owner_hash() const noexcept;`
- `bool weak_ptr::owner_equal(...) const noexcept;`
- `template <class T> struct owner_hash;`
- `template <class T> struct owner_equal;`

`shared_ptr` and `weak_ptr` would gain two new methods each: `owner_hash` and `owner_equal`. `owner_hash` would return a ownership based hash and `owner_equal` would compare ownership.

In addition, two new class templates would be provided: `owner_hash` and `owner_equal`.

This option has the benefit of providing all the tools necessary for having a `weak_ptr` key in an unordered associative container.

The addition of the methods and template specializations for `shared_ptr` enables the ability to look up values in an unordered associative container from either the `weak_ptr` or the corresponding `shared_ptr`. It also enables storing `shared_ptr` keys based on the ownership equivalence relation, if such a thing is desired.

I don't believe this proposal exposes any more implementation details than are already exposed through `owner_less`.

To satisfy the *Cpp17Hash* requirements that the probability of a collision be low, a conforming implementation could simply hash the address of the control block, per `hash<void*>`.

Alternatives Considered

`owner_ptr` / `owner` / `owner_id`

This option would instead add the following methods:

- `const unspecified shared_ptr::owner_ptr() const noexcept;`
- `const unspecified weak_ptr::owner_ptr() const noexcept;`

The *unspecified* return type would need to be something for which there is, or could be, a `std::hash` specialization and an equality comparator (for example, a pointer to the control block, a guid stored in the control block, etc.).

This option would look to provide the bare minimum required, leaving it up to the user to define `owner_hash` and `owner_equal` in terms of this method.

`shared_ptr_key` / `weak_ptr_key`

This option would provide wrapper classes, `shared_ptr_key` and `weak_ptr_key`, which would have a `std::hash` specialization and an equality comparator. They would presumably be friends of `shared_ptr` and `weak_ptr` respectively.

Do Nothing

Whilst writing this paper, I discovered that it is actually possible for end users to do this right now in terms of `owner_before`, it just wouldn't be efficient.

`owner_hash` would be implemented to return a constant value — completely going against the *Cpp17Hash* requirements which state that the probability of a collision be low.

and

`owner_equal(p, q)` would be implemented to return `!p.owner_before(q) && !q.owner_before(p)`.

Wording

Class template `owner_hash` [util.smartptr.ownerhash]

Add this entire section.

The class template `owner_hash` allows ownership-based hashing.

```
namespace std {
    template <class T> struct owner_hash;

    template <class T> struct owner_hash<shared_ptr<T>> {
        size_t operator()(const shared_ptr<T>&) const noexcept;
    };

    template <class T> struct owner_hash<weak_ptr<T>> {
        size_t operator()(const weak_ptr<T>&) const noexcept;
    };
}
```

`operator()(x)` shall return `x.owner_hash()`.

Class template `owner_equal` [util.smartptr.ownerequal]

Add this entire section.

The class template `owner_equal` allows ownership-based mixed equality comparisons of shared and weak pointers.

```
namespace std {
    template <class T = void> struct owner_equal;

    template <class T> struct owner_equal<shared_ptr<T>> {
        bool operator()(const shared_ptr<T>&, const shared_ptr<T>&) const noexcept;
        bool operator()(const shared_ptr<T>&, const weak_ptr<T>&) const noexcept;
        bool operator()(const weak_ptr<T>&, const shared_ptr<T>&) const noexcept;
    };

    template <class T> struct owner_equal<weak_ptr<T>> {
        bool operator()(const weak_ptr<T>&, const weak_ptr<T>&) const noexcept;
        bool operator()(const shared_ptr<T>&, const weak_ptr<T>&) const noexcept;
        bool operator()(const weak_ptr<T>&, const shared_ptr<T>&) const noexcept;
    };

    template <> struct owner_equal<void> {
        template <class T, class U>
            bool operator()(const shared_ptr<T>&, const shared_ptr<U>&) const noexcept;
        template <class T, class U>
            bool operator()(const shared_ptr<T>&, const weak_ptr<U>&) const noexcept;
        template <class T, class U>
            bool operator()(const weak_ptr<T>&, const shared_ptr<U>&) const noexcept;
        template <class T, class U>
            bool operator()(const weak_ptr<T>&, const weak_ptr<U>&) const noexcept;

        using is_transparent = unspecified;
    };
}
```

```
};
}
```

`operator()(x, y)` shall return `x.owner_equal(y)`.

Class template `shared_ptr` [util.smartptr.shared]

Add to the observers section:

```
size_t owner_hash() const noexcept;
template<class U>
    bool owner_equal(const shared_ptr<U>& b) const noexcept;
template<class U>
    bool owner_equal(const weak_ptr<U>& b) const noexcept;
```

Observers [util.smartptr.shared.obs]

Add at the end:

```
size_t owner_hash() const noexcept;
```

Returns: An unspecified value such that

— `x.owner_hash() == y.owner_hash()` is true if `x.owner_equal() == y.owner_equal()`

```
template<class U>
    bool owner_equal(const shared_ptr<U>& b) const noexcept;
template<class U>
    bool owner_equal(const weak_ptr<U>& b) const noexcept;
```

Returns: true if `*this` and `b` share ownership or are both empty.

Class template `weak_ptr` [util.smartptr.weak]

Add to the observers section:

```
size_t owner_hash() const noexcept;
template<class U>
    bool owner_equal(const shared_ptr<U>& b) const noexcept;
template<class U>
    bool owner_equal(const weak_ptr<U>& b) const noexcept;
```

Observers [util.smartptr.weak.obs]

Add at the end:

```
size_t owner_hash() const noexcept;
```

Returns: An unspecified value such that

— `x.owner_hash() == y.owner_hash()` is true if `x.owner_equal() == y.owner_equal()`

```
template<class U>
    bool owner_equal(const shared_ptr<U>& b) const noexcept;
template<class U>
    bool owner_equal(const weak_ptr<U>& b) const noexcept;
```

Returns: true if `*this` and `b` share ownership or are both empty.

Class template `enable_shared_from_this` [util.smartptr.enab]

It might make sense to change the example assert from

```
assert(!p.owner_before(q) && !q.owner_before(p)); // p and q share ownership
```

to

```
assert(p.owner_equal(q)); // p and q share ownership
```

References

- LWG Issue 1406 — <http://www.open-std.org/jtc1/sc22/wg21/docs/lwg-closed.html#1406>
- N2637 Revisiting `std::shared_ptr` comparison — <https://wg21.link/n2637>
- *Cpp17Hash* requirements — [tab:cpp17.hash] in <https://wg21.link/n4830>

Acknowledgements

Thanks to Alexander Jones, Alisdair Meredith and Masud Rahman for their help reviewing, suggesting alternatives, and providing background on the history of `owner_less`.